

PRACTICA 3 - IA

En esta práctica hemos creado un Agente para el juego del 3 en ralla pero mejorado con varios modos.

AgenteEstudiante.cpp

Este es el archivo principal que hemos usado, despues tenemos el .hpp, pero es solo una plantilla.

Principalmente vamos a comentar las funciones importantes que hemos creado

FUNCION STATUS

En esta funcion se hace una busqueda completa del arbol del juego, para buscar la victoria/empate o si terminará perdiendo.

Primero comprueba que no sucede ninguna de las situaciones anteriores y genera y analiza los movimientos posibles recursivamente.

```
AgenteEstudiante::Resultado AgenteEstudiante::Status(const Tablero &tablero,
pair<int,int> &Mov) {
    /* ===== Este trozo de código se tiene que quedar aquí
===== */
    nodosVisitados++;
    /* ===== Empieza a partir de aquí tu implementación =====
*/

    Mov = {-1, -1};

    int oponente = (id == 1) ? 2 : 1;

    // Comprobamos si el tablero actual ya es terminal.
    int ganador = tablero.comprobarGanador();

    if (ganador == id) {
        return Resultado::VICTORIA;
    }

    if (ganador == oponente) {
        return Resultado::DERROTA;
    }

    if (ganador == -1) {
        return Resultado::EMPATE;
    }

    // Generamos sucesores con sus movimientos asociados.
    auto sucesores = tablero.getSucesoresConMovimientos();

    // Si no hay sucesores, tratamos la posición como empate.
```

```
if (sucesores.empty()) {
    return Resultado::EMPATE;
}

bool mueveNuestroJugador = (tablero.getJugadorTurno() == id);

if (mueveNuestroJugador) {
    // Nuestro jugador intenta maximizar:
    // VICTORIA > EMPATE > DERROTA

    Resultado mejorResultado = Resultado::DERROTA;
    pair<int,int> mejorMov = sucesores[0].second;

    for (const auto& sucesor : sucesores) {

        pair<int,int> movHijo;
        Resultado r = Status(sucesor.first, movHijo);

        if (r == Resultado::VICTORIA) {
            Mov = sucesor.second;
            return Resultado::VICTORIA;
        }

        if (r == Resultado::EMPATE && mejorResultado == Resultado::DERROTA) {
            mejorResultado = Resultado::EMPATE;
            mejorMov = sucesor.second;
        }

    }

    Mov = mejorMov;
    return mejorResultado;
}

else {
    // El rival intenta minimizar nuestro resultado:
    // para nosotros: DERROTA < EMPATE < VICTORIA

    Resultado peorResultado = Resultado::VICTORIA;
    pair<int,int> peorMov = sucesores[0].second;

    for (const auto& sucesor : sucesores) {
        pair<int,int> movHijo;
        Resultado r = Status(sucesor.first, movHijo);

        if (r == Resultado::DERROTA) {
            Mov = sucesor.second;
            return Resultado::DERROTA;
        }

        if (r == Resultado::EMPATE && peorResultado == Resultado::VICTORIA) {
            peorResultado = Resultado::EMPATE;
            peorMov = sucesor.second;
        }

    }
}
```

```

        Mov = peorMov;
        return peorResultado;
    }
}

```

FUNCION MINIMAX

En esta empleamos el algoritmo minimax clásico, explora las jugadas por profundidad y elige el movimiento mejor valorado.

Primero comprobamos que el tablero no tiene ganador (si lo tiene devuelve un valor muy alto) o si gana el rival (valor muy pequeño) y empate (devuelve 0), si no se da ninguna de estas se alcanza la máxima profundidad y valoramos.

```

double AgenteEstudiante::minimax(const Tablero &tablero, int profundidad, int
prof_Max, std::pair<int,int> &Mov) {
    /* ===== Este trozo de código se tiene que quedar aquí
===== */
    nodosVisitados++;
    if (abortarBanda) return 0;

    if (std::chrono::duration<double>(std::chrono::steady_clock::now() -
inicioBusqueda).count() > tiempoMaxSegundos) {
        abortarBanda = true;
        return 0;
    }
    /* ===== Empieza a partir de aquí tu implementación =====
*/

    Mov = {-1, -1};

    int oponente = (id == 1) ? 2 : 1;

    // 1. Comprobar si el tablero actual ya es terminal.
    Tablero copia = tablero;
    int ganador = copia.comprobarGanador();

    if (ganador == id) {
        return GANAR - profundidad;
    }

    if (ganador == oponente) {
        return PERDER + profundidad;
    }

    if (ganador == -1) {
        return 0;
    }
}

```

```
// 2. Si hemos llegado a la profundidad máxima, usamos la heurística.
if (profundidad >= prof_Max) {
    return heuristica(tablero);
}

// 3. Generar sucesores.
auto sucesores = tablero.getSucesoresConMovimientos();

if (sucesores.empty()) {
    return heuristica(tablero);
}

bool mueveNuestroJugador = (tablero.getJugadorTurno() == id);

if (mueveNuestroJugador) {
    // Nodo MAX: nuestro agente quiere maximizar la puntuación.
    double mejorValor = MenosInfinito;
    std::pair<int,int> mejorMov = sucesores[0].second;

    for (const auto& sucesor : sucesores) {
        std::pair<int,int> movHijo;

        double valor = minimax(sucesor.first, profundidad + 1, prof_Max,
movHijo);

        if (valor > mejorValor) {
            mejorValor = valor;
            mejorMov = sucesor.second;
        }
    }

    Mov = mejorMov;
    return mejorValor;
}
else {
    // Nodo MIN: el rival intenta minimizar nuestra puntuación.
    double mejorValor = MasInfinito;
    std::pair<int,int> mejorMov = sucesores[0].second;

    for (const auto& sucesor : sucesores) {
        std::pair<int,int> movHijo;

        double valor = minimax(sucesor.first, profundidad + 1, prof_Max,
movHijo);

        if (valor < mejorValor) {
            mejorValor = valor;
            mejorMov = sucesor.second;
        }
    }

    Mov = mejorMov;
    return mejorValor;
}
```

```
}
}
```

FUNCION ALFABETA

Es una mejora de minimax con poda Alfa-Beta, evitando explorar ramas no necesarias y haciendolo algo más eficiente. Y se basa en buscar la mejor opcion tanto para el jugador que maximiza como para el que minimiza (Alfa y Beta respectivamente).

Si alfa es mayor o igual que beta se corta la rama, ya que sabemos que la otra es más eficiente.

Además incluye una ordenación de sucesores (sort) para ordenar valores antes de analizar, simplemente ayuda a podar ramas antes.

```
double AgenteEstudiante::alfaBeta(const Tablero &tablero, int profundidad, int
prof_Max, double alfa, double beta, std::pair<int,int> &Mov) {
    /* ===== Este trozo de código se tiene que quedar aquí
===== */
    nodosVisitados++;
    if (abortarBanda) return 0;

    if (std::chrono::duration<double>(std::chrono::steady_clock::now() -
inicioBusqueda).count() > tiempoMaxSegundos) {
        abortarBanda = true;
        return 0;
    }
    /* ===== Empieza a partir de aquí tu implementación =====
    */

    Mov = {-1, -1};

    int oponente = (id == 1) ? 2 : 1;

    // 1. Comprobar si el tablero actual ya es terminal.
    Tablero copia = tablero;
    int ganador = copia.comprobarGanador();

    if (ganador == id) {
        return GANAR - profundidad;
    }

    if (ganador == oponente) {
        return PERDER + profundidad;
    }

    if (ganador == -1) {
        return 0;
    }
}
```

```
// 2. Si llegamos a la profundidad máxima, usamos la heurística.
if (profundidad >= prof_Max) {
    return heuristica(tablero);
}

// 3. Generar sucesores.
auto sucesores = tablero.getSucesoresConMovimientos();

if (sucesores.empty()) {
    return heuristica(tablero);
}

bool mueveNuestroJugador = (tablero.getJugadorTurno() == id);

// Ordenación de sucesores para mejorar la poda Alfa-Beta.
// Si mueve nuestro jugador, interesan primero los tableros con mayor
heurística.
// Si mueve el rival, interesan primero los tableros con menor heurística.
std::sort(sucesores.begin(), sucesores.end(),
    [&](const auto& a, const auto& b) {
        double valorA = heuristica(a.first);
        double valorB = heuristica(b.first);

        if (mueveNuestroJugador) {
            return valorA > valorB;
        }
        else {
            return valorA < valorB;
        }
    });

if (mueveNuestroJugador) {
    // Nodo MAX: nuestro agente intenta maximizar.
    double mejorValor = MenosInfinito;
    std::pair<int,int> mejorMov = sucesores[0].second;

    for (const auto& sucesor : sucesores) {
        std::pair<int,int> movHijo;

        double valor = alfaBeta(
            sucesor.first,
            profundidad + 1,
            prof_Max,
            alfa,
            beta,
            movHijo
        );

        if (valor > mejorValor) {
            mejorValor = valor;
            mejorMov = sucesor.second;
        }
    }
}
```

```
        alfa = std::max(alfa, mejorValor);

        if (alfa >= beta) {
            break;
        }
    }

    Mov = mejorMov;
    return mejorValor;
}
else {
    // Nodo MIN: el rival intenta minimizar nuestra puntuación.
    double mejorValor = MasInfinito;
    std::pair<int,int> mejorMov = sucesores[0].second;

    for (const auto& sucesor : sucesores) {
        std::pair<int,int> movHijo;

        double valor = alfaBeta(
            sucesor.first,
            profundidad + 1,
            prof_Max,
            alfa,
            beta,
            movHijo
        );

        if (valor < mejorValor) {
            mejorValor = valor;
            mejorMov = sucesor.second;
        }

        beta = std::min(beta, mejorValor);

        if (alfa >= beta) {
            break;
        }
    }

    Mov = mejorMov;
    return mejorValor;
}
}
```

FUNCION HEURISTICA

Es simplemente un selector de heuristica en base a los parámetros pasados.

```
double AgenteEstudiante::heuristica(const Tablero& tablero) {
```

```
switch(numHeuristica) {  
    case 0: return heuristicaPrueba(tablero);  
           break;  
    case 1: return heuristica1(tablero);  
           break;  
    case 2: return heuristica2(tablero);  
           break;  
    default: return heuristica1(tablero);  
}  
}
```

FUNCION HEURISTICA1

Es una función que valora si un tablero es bueno o malo para el jugador cuando no se puede explorar la partida completa.

Analiza las posibles líneas para ganar (horizontales, verticales y diagonales), si hay fichas nuestras suma puntos, si hay del rival resta, y también tiene valora las posibilidades cercanas a la victoria, union de 4 por ejemplo, y añade bonus por controlar posiciones centrales, ya que da más posibilidades.

```
double AgenteEstudiante::heuristica1(const Tablero& tablero) {  
    int filas = tablero.getFilas();  
    int columnas = tablero.getColumnas();  
    int n = tablero.getNParaGanar();  
    int oponente = (id == 1) ? 2 : 1;  
  
    // 1. Comprobación de estados terminales.  
    Tablero copia = tablero;  
    int ganador = copia.comprobarGanador();  
  
    if (ganador == id) {  
        return GANAR;  
    }  
  
    if (ganador == oponente) {  
        return PERDER;  
    }  
  
    if (ganador == -1) {  
        return 0;  
    }  
  
    double score = 0.0;  
  
    // 2. Pequeño bonus por ocupar zonas centrales.  
    // En juegos de alineación, el centro suele ser más valioso porque participa  
    // en más líneas horizontales, verticales y diagonales.  
    int centroF = filas / 2;  
    int centroC = columnas / 2;
```



```
for (int f = 0; f < filas; f++) {
    for (int c = 0; c < columnas; c++) {
        int celda = tablero.getCelda(f, c);

        if (celda != 0) {
            int valorCentro = (filas - std::abs(f - centroF)) +
                               (columnas - std::abs(c - centroC));

            if (celda == id) {
                score += 3.0 * valorCentro;
            }
            else if (celda == oponente) {
                score -= 3.0 * valorCentro;
            }
        }
    }
}

// 3. Función auxiliar para puntuar una ventana de n casillas.
auto evaluarVentana = [&](int propias, int rivales) -> double {
    // Si hay fichas de ambos jugadores, esa línea está bloqueada.
    if (propias > 0 && rivales > 0) {
        return 0.0;
    }

    // Línea vacía: no aporta información.
    if (propias == 0 && rivales == 0) {
        return 0.0;
    }

    // Línea favorable a nuestro jugador.
    if (propias > 0) {
        if (propias >= n) {
            return GANAR / 10.0;
        }

        if (propias == n - 1) {
            return 200000.0;
        }

        if (propias == n - 2) {
            return 10000.0;
        }

        if (propias == n - 3) {
            return 500.0;
        }

        return 20.0 * propias;
    }

    // Línea favorable al rival.
    if (rivales > 0) {
```

```
        if (rivales >= n) {
            return PERDER / 10.0;
        }

        if (rivales == n - 1) {
            return -25000.0;
        }

        if (rivales == n - 2) {
            return -15000.0;
        }

        if (rivales == n - 3) {
            return -700.0;
        }

        return -25.0 * rivales;
    }

    return 0.0;
};

// 4. Direcciones que vamos a analizar:
// horizontal, vertical, diagonal descendente y diagonal ascendente.
const int df[4] = {0, 1, 1, -1};
const int dc[4] = {1, 0, 1, 1};

for (int f = 0; f < filas; f++) {
    for (int c = 0; c < columnas; c++) {
        for (int dir = 0; dir < 4; dir++) {
            int finF = f + (n - 1) * df[dir];
            int finC = c + (n - 1) * dc[dir];

            // Comprobamos que la ventana cabe dentro del tablero.
            if (finF < 0 || finF >= filas || finC < 0 || finC >= columnas) {
                continue;
            }

            int propias = 0;
            int rivales = 0;

            for (int k = 0; k < n; k++) {
                int nf = f + k * df[dir];
                int nc = c + k * dc[dir];

                int celda = tablero.getCelda(nf, nc);

                if (celda == id) {
                    propias++;
                }
                else if (celda == oponente) {
                    rivales++;
                }
            }
        }
    }
}
```

```

        score += evaluarVentana(propias, rivales);
    }
}
}

return score;
}

```

FUNCION HEURISTICA2

Es una heurística alternativa algo más sencilla, basada en la comparación de resultados. Usa las combinaciones dadas de la clase Tablero y valora las líneas propias y penaliza las del rival dando peso dependiendo de la longitud de las cadenas.

```

double AgenteEstudiante::heuristica2(const Tablero& tablero) {
    int n = tablero.getNParaGanar();
    int oponente = (id == 1) ? 2 : 1;

    // Comprobación de estados terminales.
    Tablero copia = tablero;
    int ganador = copia.comprobarGanador();

    if (ganador == id) {
        return GANAR;
    }

    if (ganador == oponente) {
        return PERDER;
    }

    if (ganador == -1) {
        return 0;
    }

    double score = 0.0;

    // Valoramos combinaciones propias y del rival.
    // Cuanto más largas sean las líneas, más peso tienen.
    for (int longitud = 1; longitud <= n; longitud++) {
        int propias = tablero.contarCombinaciones(longitud, id);
        int rivales = tablero.contarCombinaciones(longitud, oponente);

        double peso = std::pow(10.0, longitud);

        score += peso * propias;
        score -= peso * 1.25 * rivales;
    }
}

```

```

// Pequeño bonus por control del centro.
int filas = tablero.getFilas();
int columnas = tablero.getColumnas();
int centroF = filas / 2;
int centroC = columnas / 2;

for (int f = 0; f < filas; f++) {
    for (int c = 0; c < columnas; c++) {
        int celda = tablero.getCelda(f, c);

        if (celda != 0) {
            int distancia = std::abs(f - centroF) + std::abs(c - centroC);
            double valorCentro = 10.0 - distancia;

            if (celda == id) {
                score += valorCentro;
            }
            else if (celda == oponente) {
                score -= valorCentro;
            }
        }
    }
}

return score;
}

```

FUNCION JUEGAINTELIGENTE

Se usa cuando el agente se ejecuta en modo inteligente. Y hace llamadas a Alfa-Beta, una vez devuelve el movimiento elegido y el simulador coloca la ficha. Es importante ya que conecta el simulador con nuestra implementación real de búsqueda inteligente.

```

pair<int, int> AgenteEstudiante::JuegaInteligente(const Tablero& tablero) {
    pair<int, int> Mov;

    double valor = alfaBeta(tablero, 0, profundidadMax, MenosInfinito,
MasInfinito, Mov);
    cout << "Valor Minimax: " << valor << "\tJugada: (" << Mov.first << ", " <<
Mov.second << ")\n";
    return Mov;
}

```